

Advanced JavaScript Lab

An advanced deep dive into Asynchronous Patterns, Error Handling, and the magic of Async/Await.

Ionescu Andrei

The Problem: Blocking IO

Network requests take time

Browsers are single-threaded. If we wait for a request synchronously, the UI freezes.

- **Promises:** Represent a value that will exist "later".
- **Fetch:** Returns a promise immediately.
- **Non-blocking:** The script keeps running while the data travels.



The async Function



What it does

Prefixing a function with `async` ensures it always returns a **Promise**.

Even if you return a simple value, JavaScript wraps it in a `Promise.resolve()` automatically.

```
async function myApp() {  
  return "Hello!";  
}  
  
// myApp() returns a Promise  
myApp().then(val => console.log(val));
```

The await Magic

Pausing Local Execution

The await keyword makes JavaScript wait until the Promise settles (resolves or rejects).

Only works inside async functions.

It **pauses** the function, not the whole thread.

Result is the resolved value, not the Promise itself.



The try...catch Block

Safety First

When an awaited promise fails (network down, 404), it throws an exception.

Without try...catch, your entire script crashes.

The catch(error) block lets you handle failures gracefully (e.g., showing an alert).



The Full Pattern

Combining async, await, and try...catch

The **complete boilerplate** for making modern, safe network requests in JavaScript.

Ensures your code is readable (looks synchronous) while remaining non-blocking.

Mandatory pattern for production apps to prevent silent failures and crashes.

```
async function fetchData() {  
  try {  
    // 1. Start the request  
    const response = await fetch(url);  
    // 2. Parse the result  
    const data = await  
response.json();  
    console.log(data);  
  } catch (error) {  
    // 3. Handle failures  
    console.error('Failed:', error);  
  }  
}
```

Headers & Security

Beyond the URL

Real-world APIs often require authentication and specific content types.

- **API Keys:** Passed in Authorization headers.
- **Content-Type:** Telling the server you are sending JSON.
- **CORS:** Understanding cross-origin security policies.

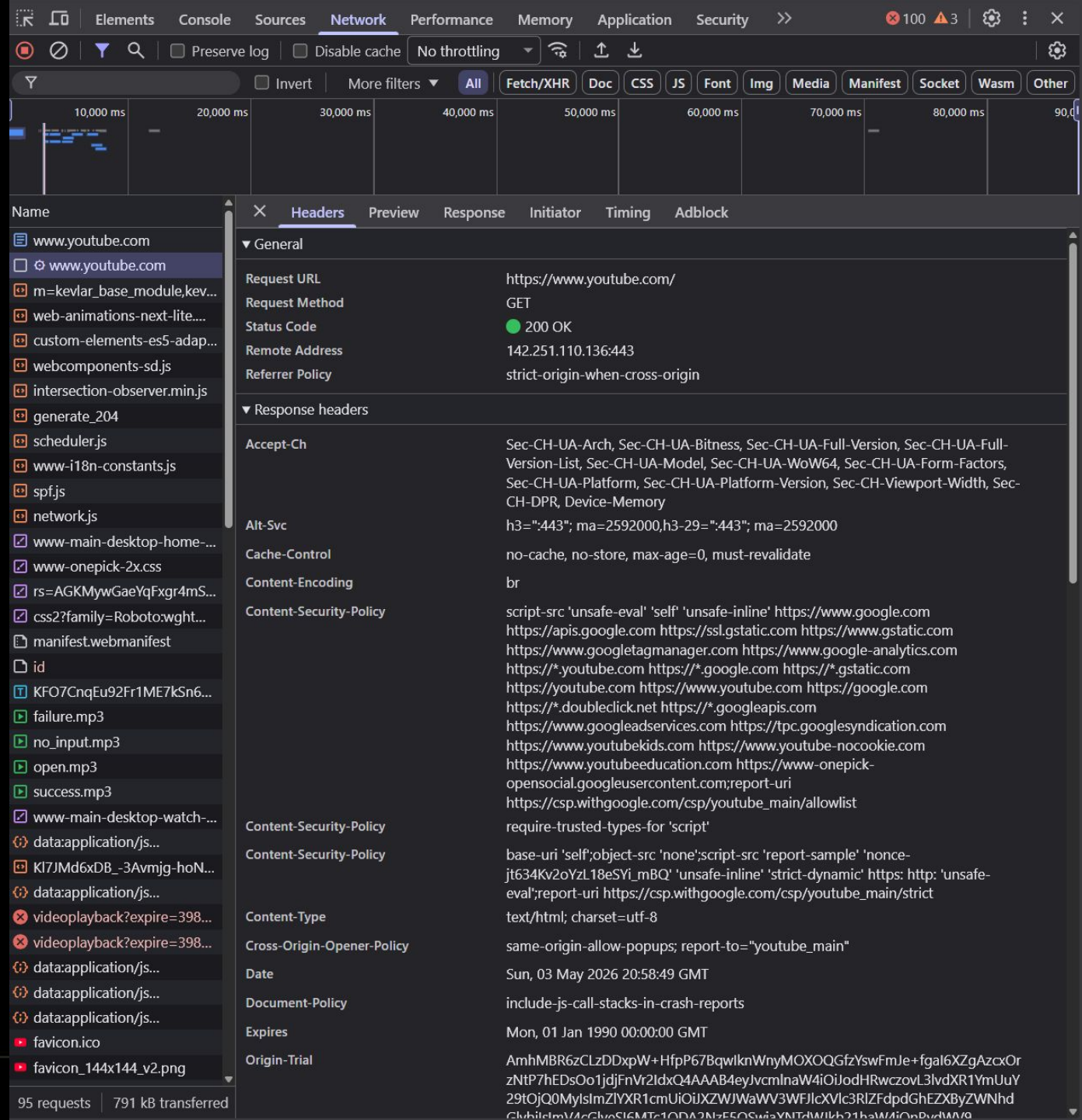
```
headers: {  
  'Content-Type': 'application/json',  
  'Authorization': 'Bearer KEY'  
}
```

Under the Hood

The Network Tab

Your best friend in debugging. Learn to inspect:

- **Payload:** What did you actually send?
- **Preview:** What did the server send back?
- **Timing:** Why is the request taking 2 seconds?



Data Transformation

Filter & Map

APIs give you **everything**. You only need **some** things.

Learn to clean your data immediately after parsing the JSON to keep your UI logic simple and efficient.

```
// Extracting only what we need  
const cleanData = data.results.map(p => ({  
  name: p.name,  
  url: p.url  
}));
```

Questions?

Ready to fetch some data? Your laboratory starts now.

<https://github.com/ionescuaandrei/AdvancedJSLab-FILS>

Feedback Time

